Resume Explanation: Credit Card Default Prediction

This document walks through your resume bullet point by point, phrase by phrase. For each piece, it explains **what it means**, **what was actually built** (with real numbers from your project), and **how to say it out loud** in an interview.

Your resume text, for reference:

> ***Credit Card Default Prediction*** *- Built an ML model to predict credit card defaults on a 30K-customer dataset, performing data cleaning, EDA with Pandas, Matplotlib and Seaborn, and creating features such as credit utilization ratio and number of delayed payments. - Trained and compared Logistic Regression, Decision Tree and Random Forest classifiers with scikit-learn, using a train-test split and GridSearchCV for basic hyperparameter tuning. - Evaluated models with accuracy, precision, recall, F1-score and confusion matrix, and handled class imbalance (22% defaulters) using class weights, since accuracy alone was misleading. - Deployed the final model as a FastAPI REST API with Pydantic input validation, returning the default probability and risk category.*

Bullet 1: Data, Cleaning, EDA, and Features

"Built an ML model to predict credit card defaults on a 30K-customer dataset"

What it means: The target problem — binary classification. Given a customer's credit profile, predict whether they will default on their payment next month.

What was actually built: Trained on the UCI "Default of Credit Card Clients" dataset — a real, public dataset of 30,000 customers from Taiwan, 22.12% of whom defaulted. Not synthetic data — this is the exact number on the resume because the dataset really has 30,000 rows and a 22% default rate.

How to say it: *"I used a real public dataset — the UCI credit card default dataset — 30,000 customers with a 22% default rate, so it's a realistically imbalanced classification problem, not a toy example."*

"performing data cleaning"

What it means: Fixing data quality issues before any modeling.

What was actually built: Two columns, `EDUCATION` and `MARRIAGE`, have category codes that aren't documented. `EDUCATION` is supposed to only be 1 (grad school), 2 (university), 3 (high school), or 4 (other) — but the raw data also contains 0, 5, and 6. `MARRIAGE` is supposed to only be 1 (married), 2 (single), or 3 (other) — but the raw data also contains 0. These undocumented codes were folded into the existing "other" category (`src/data_prep.py::clean_undocumented_codes`) instead of being dropped, which keeps those rows in the dataset instead of throwing away real data over one bad column.

How to say it: *"The EDUCATION and MARRIAGE columns had category codes outside what the data dictionary documented — 0, 5, 6 for EDUCATION, 0 for MARRIAGE. I folded those into the existing 'other' bucket rather than dropping the rows, since they're almost certainly data-entry artifacts, not real categories, and dropping them would've thrown away a few hundred otherwise-valid customers."*

"EDA with Pandas, Matplotlib and Seaborn"

What it means: Exploratory Data Analysis — using code (not a BI tool) to understand the data's structure and relationships before deciding what features to build or which models make sense.

What was actually built (`src/eda.py`, plots saved to `reports/figures/`): - **Class balance** — confirmed the 78%/22% split, which is the reason class weights were needed later (see Bullet 3). - **Default rate by education level** — default rate ranges from ~19% (grad school) up to ~25% (high school), confirming EDUCATION is actually predictive. - **Credit utilization histogram by default status** — defaulters visibly cluster toward higher utilization (near or at their credit limit); non-defaulters cluster near low utilization. This is the direct justification for engineering the utilization-ratio feature. - **Default rate by number of delayed payments** — the single strongest chart in the project: default rate rises almost linearly from 12% (zero delayed months) to 70% (six delayed months). - **Correlation heatmap** — turns the visual patterns into numbers: `NUM_DELAYED_PAYMENTS` correlates with default at **+0.40** (by far the strongest), `LIMIT_BAL` at **-0.15**, `CREDIT_UTIL_RATIO` at **+0.09**.

How to say it: *"Before building any features, I used Pandas to profile the data and Matplotlib/Seaborn to visualize it — class balance, default rate broken down by category, and a correlation heatmap. That's what told me which features were actually worth engineering, rather than guessing."*

"creating features such as credit utilization ratio and number of delayed payments"

What it means: Turning raw columns into new, more informative inputs for the model.

What was actually built (`src/features.py`): - `CREDIT_UTIL_RATIO = BILL_AMT1 / LIMIT_BAL` — how much of their credit limit the customer is currently using. A raw bill of $50,000 means something very different for someone with a $60,000 limit versus a $500,000 limit; this ratio normalizes for that. - `NUM_DELAYED_PAYMENTS = count of PAY_1..PAY_6 > 0` — collapses six separate monthly repayment-status columns into one number: how many of the last 6 months had a late payment.

Both were chosen *because* the EDA plots above showed they separate defaulters from non-defaulters well — not guessed at in advance.

How to say it: *"I didn't start with feature engineering — I started with EDA, saw that raw utilization and repayment delays visually separated the two classes, and then built features around exactly that. Utilization ratio normalizes bill amount by credit limit; delayed-payments count turns 6 monthly status columns into one interpretable number."*

Bullet 2: Model Training

"Trained and compared Logistic Regression, Decision Tree and Random Forest classifiers with scikit-learn"

What it means: Comparing three models with different underlying assumptions instead of committing to just one.

What was actually built (`src/train.py`), with real test-set results:

Model	Accuracy	Precision	Recall	F1
Logistic Regression	76.3%	0.470	0.561	0.511
Decision Tree	75.5%	0.457	0.586	0.514
Random Forest (winner)	**77.5%**	**0.492**	**0.608**	**0.544**

- **Logistic Regression** draws a single linear boundary between the classes — fast, interpretable, and the standard baseline every other model needs to beat.
- **Decision Tree** asks a sequence of yes/no questions on the features (e.g. "is NUM_DELAYED_PAYMENTS > 2?") and can capture non-linear patterns a straight line can't.
- **Random Forest** trains 400 decision trees on random subsets of the data and features, then averages their predictions — captures the same non-linearity as a single tree, but far less prone to overfitting any one quirk of the training data, which is why it won on every metric.

How to say it: *"I compared three models that make different assumptions about the data — linear, single non-linear tree, and an ensemble of trees — because you can't know in advance which fits best. Random Forest won on every metric: 77.5% accuracy, F1 of 0.544, versus 0.511 and 0.514 for the other two."*

"using a train-test split and GridSearchCV for basic hyperparameter tuning"

What it means: Holding out data the model never trains on to get an honest read of performance, and systematically searching for the best hyperparameters instead of guessing at them.

What was actually built: An 80/20 train/test split, stratified so both splits keep the same ~22% default rate. `GridSearchCV(cv=5)` on each model's pipeline tries every combination in a small hyperparameter grid (e.g. Random Forest: `n_estimators` in [200, 400], `max_depth` in [6, 10, None]) across 5 cross-validation folds and keeps whichever generalizes best. The winning hyperparameters: Logistic Regression `C=0.01`, Decision Tree `max_depth=4, min_samples_leaf=50`, Random Forest `max_depth=10, n_estimators=400`.

How to say it: *"I used an 80/20 stratified train-test split so the held-out set has the same class balance as training, and GridSearchCV with 5-fold cross-validation to tune each model's hyperparameters instead of using library defaults — it tries every combination in a small grid and picks whichever scores best, averaged across folds, not just on one lucky split."*

Bullet 3: Evaluation and Class Imbalance

"Evaluated models with accuracy, precision, recall, F1-score and confusion matrix"

What it means: Using multiple complementary metrics instead of one number, because different metrics expose different kinds of mistakes.

What was actually built: For the winning Random Forest model, the confusion matrix on the 6,000-row test set:

	predicted: no default	predicted: default
actual: no default	3,840	833
actual: default	520	807

From this: **accuracy** = (3840+807)/6000 = 77.5%. **Precision** = 807/(807+833) = 49.2% — of everyone flagged high-risk, 49% actually defaulted. **Recall** = 807/(807+520) = 60.8% — of everyone who actually defaulted, 61% were caught. **F1** = 0.544, the harmonic mean balancing the two.

How to say it: *"The confusion matrix shows exactly how the model is wrong, not just how often. My Random Forest catches 61% of actual defaulters (recall) and is right about half the time when it flags someone as high risk (precision). Those two numbers matter more individually than the single accuracy figure, because they tell you what kind of mistake the model is making, not just how many."*

"handled class imbalance (22% defaulters) using class weights, since accuracy alone was misleading"

What it means: Only 22% of customers default, so a trivial model that always predicts "no default" would already score ~78% accuracy without being useful at all. Accuracy alone can't distinguish a genuinely good model from a lazy one.

What was actually built: `class_weight="balanced"` on every model (`clf = LogisticRegression(..., class_weight="balanced")`, same for DecisionTree and RandomForest). This tells the model to penalize mistakes on the minority (default) class more heavily during training — roughly proportional to how rare it is (~3.5x more penalty per default-class mistake, since the split is ~78/22) — instead of treating every mistake the same and defaulting to just predicting the majority class.

How to say it: *"With a 22% positive rate, a model that always says 'no default' would already hit ~78% accuracy while catching zero actual defaulters — that's why I didn't lead with accuracy. I used class_weight='balanced' on every model so misclassifying a defaulter costs the model more during training than misclassifying a non-defaulter, and I evaluated with precision/recall/F1 specifically because those expose whether the model is actually catching defaulters, not just predicting the majority class."*

Bullet 4: Deployment

"Deployed the final model as a FastAPI REST API with Pydantic input validation"

What it means: Making the trained model usable by other software over HTTP, with safeguards against bad input.

What was actually built (`api/`): A FastAPI app with a `/predict` endpoint and a `/health` endpoint. The model is loaded once at startup (not re-loaded per request) via FastAPI's lifespan hook. Every request body is validated against a Pydantic schema (`api/schemas.py`) before it ever reaches the model — field types, and realistic ranges (`age` between 18-100, `education` in 1-4, `sex` in 1-2,

payment amounts non-negative). Invalid input gets a clean `422` error instead of crashing the model or silently producing garbage. Verified with an automated `pytest` suite covering both valid predictions and each validation rule.

How to say it: *"I wrapped the trained model in a FastAPI service. Pydantic schemas validate every field's type and range before the request reaches the model, so malformed input — a negative age, an undocumented education code — gets rejected with a clear error instead of producing a silent bad prediction. The model itself loads once at startup, not on every request, and I have automated tests covering both the happy path and every validation rule."*

"returning the default probability and risk category"

What it means: The API's output format — not just a binary yes/no, but a probability plus a simple business-facing label.

What was actually built: `/predict` returns a JSON body like `{"default_probability": 0.85, "risk_category": "high_risk"}`. The raw probability comes from `model.predict_proba()`; `risk_category` is a simple threshold on top of it (`"high_risk"` if probability >= 0.5, else `"low_risk"`) so a non-technical consumer of the API doesn't have to interpret a raw float.

How to say it: *"The endpoint returns both the raw probability and a simple risk_category label. The probability is useful if a downstream system wants to set its own threshold or rank customers by risk; the category is a simpler, decision-ready output for anything that just needs a yes/no flag."*